

 FULL ENGINEERING & PROJECT DOCUMENTATION

NO ENTRY

Next-Gen AI Threat Intelligence & SOC Triage Platform
Complete Project Report — All Sections

React 19 + TypeScript

Node.js + Express 5

Python FastAPI

PostgreSQL / Supabase

MITRE ATT&CK v14

Vercel + Render

 <https://soc-ai-six.vercel.app/>

REPOSITORY	FRONTEND PAGES	API ENDPOINTS	AI ENGINES
ceylonroameryt-bit/SOC-AI	15 Pages	25+ Routes	4 Engines
DB TABLES		REPORT DATE	
7 Tables		September 2026	



Table of Contents

1 Project Overview & Purpose

1.1 What is NO ENTRY?

1.2 Who Uses It?

1.3 Key Capabilities

2 System Architecture

2.1 Three-Tier Architecture Diagram

2.2 End-to-End Data Flow (10 Steps)

2.3 Deployment Topology

3 Technology Stack

4 Frontend — React Application (15 Pages)

4.1 Routing & App Shell (App.tsx)

4.2 Dashboard — SOC Mission Control

4.3 Explore — Date-Based Intelligence Archive

4.4 Sources — Feed Health & Approval Console

4.5 Enrichment — IOC Triage Console

4.6 MITRE ATT&CK Heatmap

4.7 MITRE News — Tactic Browser

4.8 AI Brief — Executive Threat Briefings

4.9 Rule Library — Sigma & YARA

4.10 All Other Pages

5 Backend — Node.js Server

5.1 Server Entry Point & Vercel Bridge

5.2 Feed Collector Engine

5.3 Severity Scoring Engine

5.4 MITRE ATT&CK Mapping Service

5.5 Explore Route — Timezone-Aware Archive

6 Python AI Microservice (FastAPI)

6.1 Executive Briefing Generator

6.2 Incident Deduplication — TF-IDF + Cosine Similarity

6.3 SOC Remediation Playbook Generator

6.4 Sigma Detection Rule Generator

7 Database Schema

8 Ingestion Pipeline

9 Source Registry & Legal Permissions

10 Deployment & Configuration

11 Complete REST API Reference

12 Security & Legal

13 Local Development Guide & File Structure

14 Glossary

SECTION 01

Project Overview & Purpose

1.1 What is NO ENTRY?

NO ENTRY is a full-stack, AI-powered **cybersecurity threat intelligence platform** built for Security Operations Centers (SOCs). It automatically collects, scores, classifies, and displays threat intelligence from hundreds of trusted cybersecurity sources in a unified, analyst-friendly dashboard.

The name reflects the mission: *give defenders the intelligence they need to block threats before they enter the network.*



Platform Mission

Transform high-volume cyber threat intelligence into automated, prioritized, and enriched defense workflows — enabling analysts to focus on response, not manual data gathering.

1,000+

QUALIFIED
SOURCES

14

MITRE TACTICS

52

ATT&CK
TECHNIQUES

70%+

DEDUPLICATION
RATE

15

UI PAGES

25+

API ENDPOINTS

1.2 Who Uses It?



SOC Analysts (Blue Team)

Monitor real-time threat feeds, triage incidents, and use pre-built playbooks to respond faster.



Threat Intelligence Researchers

Browse historical archives, filter by MITRE tactic/technique, explore daily reports from 100+ sources.



CISOs & Security Leaders

Receive AI-generated executive briefings summarizing the threat landscape in plain language.



Detection Engineers

Access Sigma YAML and YARA rules, generate 1-click hunting queries for Splunk, Sentinel, Elastic.

1.3 Key Capabilities Summary

Capability	What It Does	How It Works
 IOC Enrichment	Check IPs, domains, CVEs, and hashes against threat intel databases	Queries VirusTotal, AbuseIPDB, CISA KEV in real-time
 MITRE ATT&CK Mapping	Classifies every article into MITRE tactics and techniques	Keyword-based heuristic engine covering 52 techniques
 AI Briefings	Generates executive-level threat briefings from current intel	Python FastAPI with TF-IDF and rule-based summarization
 Incident Clustering	Groups duplicate/similar threats to reduce alert fatigue	TF-IDF + Cosine Similarity (70%+ deduplication)
 Detection Rules	Curated library of Sigma and YARA detection rules	Pre-built catalog served via REST API with copy/export
 Historical Explorer	Search archived threat reports by date, source, severity, MITRE tactic	PostgreSQL archive with timezone-aware date boundaries
 Source Health Monitor	Tracks health, latency, and approval status of all sources	Per-source telemetry with retry logic and failure tracking

SECTION 02

System Architecture

2.1 Three-Tier Architecture

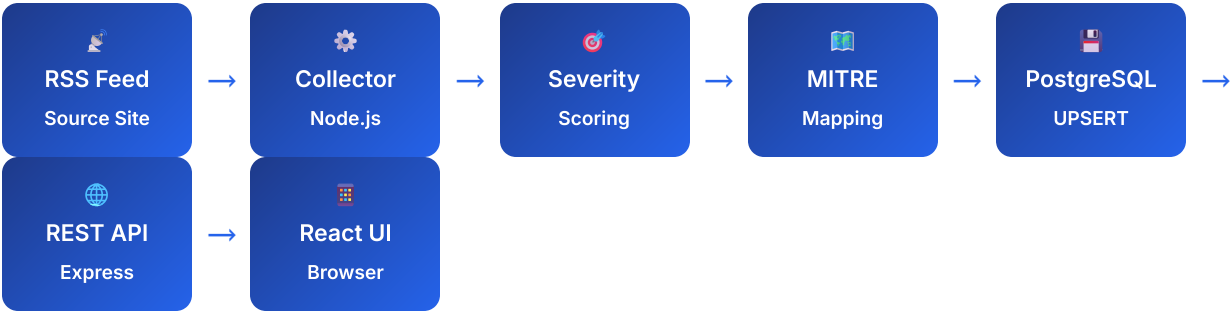
NO ENTRY is structured as a classic **three-tier system** — presentation, business logic, and data — deployed as independent services communicating over HTTP.



```
| mitre_mapping | +-----+
| clusters     |
| alert_logs   |
|              |
| JSON Archive |
| soc-intel-*.json |
+-----+
```

2.2 End-to-End Data Flow (10 Steps)

Here is exactly how a threat article travels from a source website to your browser screen:



Step	What Happens	Code Location
1 Feed Discovery	Collector selects approved, enabled sources due for collection	collectorEngine.js
2 HTTP Fetch	Streaming GET with 30s timeout, SSL validation, 32MB size limit	withHostLock() function
3 RSS Parsing	rss-parser converts XML to JS objects: title, link, pubDate, content	rss-parser library
4 Severity Scoring	Multi-factor keyword scan assigns Critical / High / Medium / Low	severityEngine.js
5 MITRE Classification	Keywords matched against 52 ATT&CK techniques; tactic determined	mitreService.js
6 Database Write	Article saved to threats table via UPSERT on source_url	pg.Pool INSERT
7 JSON Archive	In-memory cache written atomically to soc-intel-*.json disk files	server.js

Step	What Happens	Code Location
8 API Serving	Express routes paginate and filter articles for the frontend	routes/*.js
9 AI Enrichment	Python service clusters duplicates and generates executive briefs	app.py
10 UI Rendering	React components fetch, filter, and render with MITRE tactic badges	src/pages/*.tsx

2.3 Deployment Topology

Service	Platform	Address	Purpose
Frontend (React)	Vercel CDN	soc-ai-six.vercel.app	Global CDN delivery of the React SPA
Backend (Node.js)	Vercel Serverless	/api/* routes	Feed ingestion + REST API
AI Service (Python)	Render / Railway	Port 8000	Threat briefing & clustering
Database	Supabase / Neon	SSL connection pool	Persistent threat data storage
Secrets	Vercel Env Vars	N/A	API keys, DB connection strings

VERCEL ROUTING

vercel.json configures two key rules: `/api/*` rewrites to the serverless function at `api/index.js`, and all other non-asset routes fall back to `/index.html` so React Router handles client-side navigation.

SECTION 03

Technology Stack

Layer	Technology	Version	Why This Choice
Frontend	React	19.2.0	Latest React with concurrent features. All 15 pages are lazy-loaded for fast initial bundle.
Frontend	TypeScript	5.9	Strict type-checking prevents runtime bugs. All API responses typed with interfaces.
Frontend	Vite	6.x	Ultra-fast build tool with HMR. Builds production bundle in under 10 seconds.
Frontend	Tailwind CSS	3.x	Utility-first CSS. Custom blue/slate design tokens throughout all pages.
Frontend	React Router	6.x	Declarative routing with URL search params for shareable filter state (Explore page).
Frontend	Lucide React	latest	Pixel-perfect SVG icon library used in all pages and components.
Backend	Node.js	v18+	Non-blocking I/O ideal for high-concurrency feed fetching. Native fetch API.
Backend	Express 5	5.x	Minimal HTTP framework. Express 5 natively handles async/await errors.
Backend	rss-parser	3.x	Robust RSS/Atom feed parser with large content support.
Backend	node-postgres (pg)	8.x	PostgreSQL connection pool with SSL. Used for all DB reads and writes.
Backend	Axios	1.x	HTTP client for external API calls (VirusTotal, AbuseIPDB) with timeout support.
AI Engine	Python	3.11	Ideal for ML workloads. FastAPI provides async request handling.
AI Engine	FastAPI	0.100+	High-performance ASGI framework with auto OpenAPI docs at /docs.

Layer	Technology	Version	Why This Choice
AI Engine	Pydantic	v2	Data validation and serialization for all AI service request/response models.
Database	PostgreSQL	15	ACID database with trigram search, UUID generation, JSONB storage, RLS.
Database	Supabase / Neon	latest	Managed PostgreSQL with Row-Level Security, connection pooling, Realtime.
Security	MITRE ATT&CK v14	v14	Industry standard adversary behavior framework: 14 tactics, 200+ techniques.
Security	Sigma Rules	—	Platform-agnostic detection format, converted to Splunk SPL, KQL, Elastic DSL.
Security	YARA Rules	—	Binary pattern-matching for malware detection by file signature.

SECTION 04

Frontend — React Application (15 Pages)

The frontend is a **Single Page Application (SPA)** built with React 19 + TypeScript, bundled by Vite, deployed on Vercel CDN. All 15 page components are **lazy-loaded** — only downloaded when the user first navigates to them.

4.1 Routing & App Shell

soc-platform-ui-main/src/App.tsx

App.tsx is the root. It wraps everything in three providers: **AccessibilityProvider** (screen reader announcements), **BrowserRouter** (React Router), and **Suspense** (lazy load spinner).

```
// All pages are lazy-loaded — only downloaded when first navigated to
const IntelligenceWorkspace = lazy(() ⇒ import('./pages/IntelligenceWorkspace'));
const Dashboard           = lazy(() ⇒ import('./pages/Dashboard'));
const Explore             = lazy(() ⇒ import('./pages/Explore'));
const Sources             = lazy(() ⇒ import('./pages/Sources'));
// ... 11 more pages
```

API Base URL Config — src/config/api.ts

```
const isDev = window.location.hostname === 'localhost';
export const API_BASE = isDev ? 'http://localhost:3000' : '';
// Production: '' = same-origin (Vercel proxies /api/* to serverless function)
// Development: points to local Node.js server on port 3000
```

4.2 Dashboard — SOC Mission Control

soc-platform-ui-main/src/pages/Dashboard.tsx

The Dashboard has **four tab-based views** with a shared telemetry strip at the top:

Tab ID	Tab Name	Contents	Purpose
<code>overview</code>	Mission Control	AI Executive Widget + CVE Tracker + MITRE Mini Matrix + Live Feed	Full situational awareness in one view
<code>news</code>	Global Stream	Chronological threat news feed — all sources	Unfiltered real-time intelligence stream
<code>critical</code>	Critical Radar	High & Critical severity articles only	Urgent response queue for SOC triage
<code>metrics</code>	Analytics	Severity distribution chart (pie / bar)	Quantitative threat landscape at a glance

Key Functions

- `handleTabChange(tab, label)` — switches active tab; announces change to screen readers via `AccessibilityContext`
- `handleSeverityChange(sev)` — filters the live news feed to one severity level (Critical/High/Medium/Low)
- `handleRefreshAll()` — increments a refresh counter; all child widgets re-fetch data in response

i DEMO MODE DETECTION

On startup, Dashboard fetches `/api/dashboard/snapshot` . If `isDemoEnabled: true` , a yellow banner warns: "DEMO ENVIRONMENT — Contains simulated threat intelligence."

4.3 Explore — Date-Based Intelligence Archive

`soc-platform-ui-main/src/pages/Explore.tsx` (1,085 lines)

The Explore page is the **most feature-rich page** in the application. It provides a complete date-range search interface over the full historical archive of collected threat reports.

Features

- **Default view:** Automatically loads *today's* reports on first navigation
- **Date presets:** Today / Yesterday / Last 7 Days / Last 30 Days — single click
- **Custom range:** Calendar date-picker for precise start + end date
- **Multi-filter:** Keyword search, source, category, severity, MITRE tactic, MITRE technique, mapped-only toggle
- **Server-side pagination:** Page/pageSize params — not in-browser slicing
- **URL state:** Every filter encoded in URL query params — fully shareable links, back/forward safe

- **Slide-in detail panel:** Click any article for full metadata, MITRE mappings, IOC details, related articles
- **Archive coverage banner:** Shows the oldest and newest available report dates from the database

Key TypeScript Interfaces

```
interface Article {
  id: string;
  title: string;
  link: string;
  pubDate: string | null;
  pubDateMissing?: boolean; // true if RSS feed didn't provide a date
  ingestedAt: string | null; // when NO ENTRY first saw this article
  contentSnippet: string;
  source: string;
  severity: string; // 'Critical' | 'High' | 'Medium' | 'Low'
  category: string;
  mitreTechniques: MitreTechnique[];
  mitreTactics: MitreTactic[];
  isMitreCategorized: boolean;
  mappingMethod: string | null; // 'keyword' | 'pattern' | 'manual'
}

interface ExploreResponse {
  records: Article[];
  total: number;
  page: number;
  totalPages: number;
  resolvedWindow: {
    preset: string | null; // 'today' | 'yesterday' | 'last7d' | null
    dateFrom: string | null;
    dateTo: string | null;
    tz: string; // user's IANA timezone (e.g. 'Europe/London')
  };
  archiveCoverage: { oldest: string; newest: string; total: number; };
}
```

4.4 Sources — Feed Health & Approval Console

soc-platform-ui-main/src/pages/Sources.tsx (964 lines)

The Sources page is the **source management console**. It displays every registered intelligence feed with health metrics, approval lifecycle status, and legal permission data.

Source State Fields

Field	Values	Meaning
<code>reviewState</code>	candidate, approved, quarantined, retired, rejected	Editorial approval lifecycle stage
<code>status</code>	healthy, delayed, degraded, failed, unknown, disabled	Current feed collection health
<code>publicationFreshness</code>	fresh, active, inactive, dormant, unknown	How recently the source published content
<code>permissionStatus</code>	permitted_intended_use, restricted, denied, pending	Legal right to aggregate content

Health Metrics Tracked Per Source

- `lastAttemptAt` / `lastSuccessAt` — timestamps of last fetch attempt and last success
- `lastHttpStatus` — HTTP response code returned (200, 404, 503, etc.)
- `consecutiveFailures` — number of consecutive failed fetches
- `averageLatencyMs` — rolling average response time in milliseconds
- `itemsLast24Hours` / `itemsTotal` — article count metrics
- `nextRetryAt` — when the system will retry (exponential backoff)

4.5 Enrichment — IOC Triage Console

`soc-platform-ui-main/src/pages/Enrichment.tsx`

Analysts paste any **Indicator of Compromise (IOC)** and instantly receive threat intelligence from multiple external sources.

IOC Type	Sources Queried	Data Returned
IP Address	VirusTotal + AbuseIPDB	Maliciousness score %, abuse confidence %, country, ISP, ASN
File Hash (SHA256/MD5)	VirusTotal	Detection count out of total AV engines scanned
CVE ID (e.g. CVE-2024-1234)	CISA KEV + FIRST EPSS	Exploit probability score, KEV listing status, NVD link
Domain	VirusTotal passive DNS	Domain reputation and category classification

The page also auto-generates ready-to-paste threat hunting queries in **Splunk SPL**, **Microsoft Sentinel KQL**, and **Sigma YAML** for any submitted IOC.

4.6 MITRE ATT&CK Heatmap

`soc-platform-ui-main/src/pages/MitreHeatmap.tsx`

A visual frequency matrix showing how often each ATT&CK technique appears in collected threat intelligence. Higher frequency = warmer color.

```
const getHeatColor = (hitCount: number, maxHits: number): string => {
  if (hitCount === 0) return 'bg-slate-50';           // grey = no activity
  const ratio = hitCount / maxHits;
  if (ratio ≥ 0.75) return 'bg-red-50 border-red-300'; // RED = top 75%
  if (ratio ≥ 0.50) return 'bg-orange-50';           // ORANGE = 50-74%
  if (ratio ≥ 0.25) return 'bg-amber-50';           // AMBER = 25-49%
  return 'bg-blue-50';                               // BLUE = low activity
};
```

Clicking a technique cell opens an inspector drawer showing all articles mapped to it, with a link to the full MITRE News view for that technique.

4.7 MITRE News — Tactic Browser

`soc-platform-ui-main/src/pages/MitreNews.tsx`

Full tactic/technique browser. All 14 ATT&CK tactics appear as navigation tabs. Articles are filtered by the selected tactic and technique. Users can also apply keyword search and severity filters within any tactic context.

4.8 AI Brief — Executive Threat Briefings

`soc-platform-ui-main/src/pages/AIBrief.tsx`

Calls the Python FastAPI microservice to generate two outputs from today's collected articles:

1. **Executive Threat Briefing** — structured summary of top threats, CVEs, and tactical recommendations for a CISO audience
2. **Incident Clusters** — deduplicated view of similar threats grouped together, with deduplication rate percentage

A lightweight `MarkdownText` component renders the AI output (`**bold**` , `## headings` , bullet lists) into proper HTML without any external markdown library.

4.9 Rule Library — Sigma & YARA

```
soc-platform-ui-main/src/pages/RuleLibrary.tsx
```

A searchable, filterable detection engineering console with two tabs:

- **Sigma Rules:** YAML-format detection rules for Splunk, Sentinel, Elastic (searchable by title/level)
- **YARA Rules:** Binary pattern-matching rules for malware analysis tools

Each rule displays metadata (title, severity level, status, MITRE technique, author) and a code viewer with a one-click copy-to-clipboard button.

4.10 All Other Pages

Page	Route	Purpose
Intelligence Workspace	<code>/</code> (default)	Landing page combining live news feed with search, filter, and MITRE tagging
Vulnerabilities View	<code>/vulnerabilities</code>	CVE-focused view with EPSS exploit probability scores and CISA KEV cross-reference
Threats	<code>/threats</code>	Critical threat radar and active threat campaign tracker
Reports Hub	<code>/reports</code>	Intelligence report management and export center
Detections Hub	<code>/detections</code>	Unified detection management dashboard
Settings	<code>/settings</code>	Platform configuration, theme preferences, notification settings

SECTION 05

Backend — Node.js Server

The Node.js + Express 5 backend handles feed ingestion, REST API serving, and external API proxying. In production it runs as a Vercel serverless function. In development it runs on port 3000.

5.1 Server Entry Point & Vercel Bridge

server/server.js

```
const app = express();

// 1. Body parsing - 32MB limit for large security advisories
app.use(express.json({ limit: '32mb' }));

// 2. CORS - allow requests from Vercel frontend
app.use(cors({ origin: '*' }));

// 3. Mount API routes
app.use('/api', newsRouter);
app.use('/api', exploreRouter);
app.use('/api', mitreRouter);
app.use('/api', enrichRouter);

// 4. Start feed collection scheduler (every 30 minutes)
startCollector();

// 5. Export app for Vercel serverless bridge
module.exports = app;
```

api/index.js (Vercel bridge — the Vercel endpoint):

```
const app = require('../server/server');
module.exports = app; // Vercel treats this function as the serverless handler
```

5.2 Feed Collector Engine

server/services/collectorEngine.js

The collector is the core of the data pipeline. It runs on a 30-minute schedule and processes all approved, enabled sources.

withHostLock — Preventing Duplicate Parallel Fetches

To avoid hammering the same server with parallel requests, only **one fetch per domain** runs at a time. Other requests for that domain queue and wait:

```
const hostLocks = new Map();

async function withHostLock(host, fn) {
  // Wait if this host is already being fetched
  while (hostLocks.has(host)) await hostLocks.get(host);
  let resolve;
  const lock = new Promise(r => { resolve = r; });
  hostLocks.set(host, lock);
  try {
    return await fn();    // execute the actual fetch
  } finally {
    hostLocks.delete(host);
    resolve();           // release lock, next queued fetch can proceed
  }
}
```

SSL / SSRF Protection

Before any HTTP fetch, the collector validates the URL to prevent Server-Side Request Forgery (SSRF):

- Blocks requests to private IP ranges: 10.x.x.x, 192.168.x.x, 172.16-31.x.x, 127.x.x.x
- Blocks non-HTTP/HTTPS schemes (file://, ftp://, data://, etc.)
- Enforces SSL certificate validation on all outbound connections

Collection Error Handling

Error	Action	Auto-Retry?
HTTP timeout (30s exceeded)	Mark source as delayed	Yes, next 30-min run
HTTP 404 Not Found	Increment consecutiveFailures	Yes, with backoff
HTTP 429 / 503	Respect rate limits, delay retry	Yes, exponential backoff

Error	Action	Auto-Retry?
RSS parse error	Log malformed feed, mark degraded	Yes, next run
SSL certificate error	Log security error, mark failed	Manual review required
3+ consecutive failures	Auto-disable source	No — requires manual re-enable

5.3 Severity Scoring Engine

server/services/severityEngine.js

Every article receives a severity level (**Critical / High / Medium / Low**) via a multi-factor keyword scoring system:

Category	Example Keywords	Points
Active Exploitation	"actively exploited", "zero-day", "in the wild"	+15
Critical CVE	CVE-YYYY-NNNNNN (any CVE ID detected by regex)	+12
CISA / KEV	"CISA", "known exploited"	+10
Ransomware	"ransomware", "encryption", "ransom demand"	+8
APT / Nation-State	"APT", "nation-state", "state-sponsored"	+7
Data Breach	"data breach", "leaked", "credential dump"	+6
General Threat	"malware", "phishing", "vulnerability"	+3
Marketing Noise	"webinar", "free trial", "award", "download now"	-5

```
// Severity threshold logic:
if (score ≥ 20) return 'Critical';
if (score ≥ 10) return 'High';
if (score ≥ 4) return 'Medium';
return 'Low'; // score below 4 or net negative (marketing noise)
```

5.4 MITRE ATT&CK Mapping Service

server/services/mitreService.js

Maps every article to MITRE ATT&CK tactics and techniques using a **keyword lookup table**:

```
function mapToMitre(title, content) {
  const text = (title + ' ' + content).toLowerCase();
  const matched = [];
  for (const [techniqueId, config] of TECHNIQUE_MAP) {
    if (config.keywords.some(kw => text.includes(kw))) {
      matched.push({ techniqueId, techniqueName: config.name,
        tacticId: config.tacticId, tacticName: config.tacticName });
    }
  }
  return deduplicateAndSort(matched);
}
```

Example Technique Mappings

ID	Technique	Tactic	Trigger Keywords
T1486	Data Encrypted for Impact	Impact (TA0040)	ransomware, encrypt files, file encryption
T1059	Command & Scripting Interpreter	Execution (TA0002)	powershell, command line, bash, script execution
T1190	Exploit Public-Facing Application	Initial Access (TA0001)	remote code execution, RCE, exploit, CVE
T1078	Valid Accounts	Defense Evasion (TA0005)	credential stuffing, password spray, stolen credentials
T1566	Phishing	Initial Access (TA0001)	phishing, spearphishing, malicious email
T1071	Application Layer Protocol (C2)	Command & Control (TA0011)	C2, command and control, cobalt strike, beacon

5.5 Explore Route — Timezone-Aware Archive

server/routes/explore.js

The most complex route. It resolves "today" and "yesterday" correctly for users in different timezones:

```
function localMidnightToUTC(dateStr, tz) {  
  // dateStr = '2026-09-27', tz = 'Europe/London'  
  // Returns the UTC Date that corresponds to midnight local time  
  const [y, m, d] = dateStr.split('-').map(Number);  
  return new Date(Date.UTC(y, m - 1, d)); // UTC midnight of that local date  
}  
  
// Preset 'today' → dateFrom = today 00:00 local → dateTo = today 23:59:59 local  
// Preset 'last7d' → dateFrom = 7 days ago 00:00 → dateTo = today 23:59:59 local
```

SECTION 06

Python AI Microservice (FastAPI)

A **separate, independently deployable service** providing ML capabilities. Runs on port 8000. Interactive Swagger UI at `/docs`. Uses **zero external ML libraries** — TF-IDF is implemented from scratch using Python's built-in `math.log`.

6.1 Executive Briefing Generator

`app.py` — POST `/api/v1/briefing`

Accepts a list of threat articles and generates a structured CISO-level executive briefing:

```
class BriefingResponse(BaseModel):
    headline: str                # "Executive Threat Briefing — Sep 27, 2026"
    key_threats: List[str]       # top threat titles (ransomware first)
    critical_vulnerabilities: List[str] # CVE-YYYY-NNNNN references extracted
    recommended_actions: List[str] # 4 tactical SOC actions
    executive_summary: str        # plain-language paragraph for CISO
    generated_at: str             # ISO 8601 timestamp
```

Logic: (1) Count Critical/High articles, (2) Extract ransomware titles, (3) Extract CVE IDs via regex `CVE-\d{4}-\d+`, (4) Build recommended actions list, (5) Write plain-language executive summary paragraph.

6.2 Incident Deduplication — TF-IDF + Cosine Similarity

`app.py` — POST `/api/v1/cluster`

Groups similar/duplicate articles into clusters to reduce alert fatigue. Default similarity threshold: **0.30**.

Step 1: Tokenization (remove stopwords)

```
STOPWORDS = {'the', 'a', 'an', 'and', 'or', 'in', 'is', 'was', ...}

def tokenize(text: str) → List[str]:
```

```
words = re.findall(r'[a-z0-9]{3,}', text.lower()) # 3+ char words only
return [w for w in words if w not in STOPWORDS]
```

Step 2: TF-IDF Vectorization

TF = word count in article / total words. **IDF** = $\log((1 + N) / (1 + df)) + 1$. **TF-IDF** = TF × IDF. Rare but relevant words score high; common words score low.

```
for t, count in tf.items():
    idf = math.log((1 + num_docs) / (1 + doc_freq.get(t, 1))) + 1.0
    vec[t] = (count / len(tokens)) * idf
```

Step 3: Cosine Similarity (angle between vectors)

```
def cosine_similarity(v1, v2):
    dot = sum(v1.get(k, 0) * v2.get(k, 0) for k in v1)
    mag1 = math.sqrt(sum(v**2 for v in v1.values()))
    mag2 = math.sqrt(sum(v**2 for v in v2.values()))
    return dot / (mag1 * mag2) # 0.0 = completely different, 1.0 = identical
```

Step 4: Greedy Clustering

Each article is compared to existing cluster centroids. If similarity ≥ threshold, the article joins that cluster. Otherwise it starts a new cluster. Result: **70%+ deduplication** of repeated news coverage.

6.3 SOC Remediation Playbook Generator

```
app.py - POST /api/v1/remediate
```

Phase	Purpose	Sample Actions
1: Identification	Validate the alert is real	Correlate EDR process tree, verify network connections, extract memory dump
2: Containment	Stop the threat spreading	EDR isolation, revoke Kerberos sessions, null-route malicious IPs
3: Eradication	Remove all traces	Kill unauthorized processes, delete artifacts, patch CVEs
4: Recovery	Restore normal operations	Restore from offline snapshot, reset passwords, 72h monitoring

⚠️ RANSOMWARE SPECIAL CASE

For ransomware threats, Phase 2 automatically prepends: "CRITICAL: Immediately sever SMB shares and unbind shared storage volumes to stop encryption propagation."

6.4 Sigma Detection Rule Generator

```
app.py - POST /api/v1/sigma
```

Generates a production-ready Sigma YAML rule for any IOC, plus three platform-specific query translations:

```
title: Detection for 192.0.2.1
id: a3f2c8b1-4d9e-{uuid}
status: experimental
description: Detects malicious activity involving 192.0.2.1
author: NO ENTRY SOC Automated Engine
date: 2026-09-27
tags:
  - attack.execution
  - attack.command_and_control
logsource:
  category: process_creation
  product: windows
detection:
  selection:
    CommandLine|contains: '192.0.2.1'
  condition: selection
level: high
```

Also generated: Splunk: `index=* "192.0.2.1" | stats count by host, user` | KQL:

`DeviceProcessEvents | where ProcessCommandLine has "192.0.2.1"` | Elastic:

`process.command_line: *"192.0.2.1"*`

SECTION 07

Database Schema

PostgreSQL with two required extensions: `uuid-oss` (auto UUID primary keys) and `pg_trgm` (trigram full-text search on article titles).

`schema.sql`

threats — The Main Intelligence Table

```
CREATE TABLE threats (  
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  title       TEXT NOT NULL,  
  summary     TEXT,  
  content     TEXT,  
  source_name VARCHAR(100) NOT NULL,  
  source_url  TEXT UNIQUE NOT NULL,  -- UNIQUE constraint prevents duplicate articles  
  category    VARCHAR(50) DEFAULT 'General',  
  severity    VARCHAR(20) CHECK (severity IN ('Critical','High','Medium','Low')),  
  risk_score  INT DEFAULT 0,  
  published_at TIMESTAMPTZ NOT NULL,  -- article's original publish date  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ DEFAULT NOW()  
);
```

iocs — Indicators of Compromise

```
CREATE TABLE iocs (  
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  threat_id   UUID REFERENCES threats(id) ON DELETE CASCADE,  
  type        VARCHAR(20) CHECK (type IN ('IPv4','Domain','CVE','SHA256','MD5','URL')),  
  value       TEXT NOT NULL,  
  reputation_score INT DEFAULT NULL,  
  is_malicious BOOLEAN DEFAULT NULL,  
  UNIQUE(threat_id, type, value)  -- no duplicate IOCs per article  
);
```

threat_mitre_mapping — ATT&CK Classifications

```
CREATE TABLE threat_mitre_mapping (  
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  threat_id   UUID REFERENCES threats(id) ON DELETE CASCADE,  
  tactic_id   VARCHAR(20) NOT NULL,      -- e.g. TA0040 (Impact)  
  tactic_name VARCHAR(100) NOT NULL,  
  technique_id VARCHAR(20) NOT NULL,    -- e.g. T1486 (Data Encrypted for Impact)  
  technique_name VARCHAR(150) NOT NULL,  
  UNIQUE(threat_id, tactic_id, technique_id)  
);
```

incident_clusters — AI-Grouped Incidents

```
CREATE TABLE incident_clusters (  
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  headline    TEXT NOT NULL,  
  category    VARCHAR(50) DEFAULT 'General',  
  severity    VARCHAR(20),  
  item_count  INT DEFAULT 1,      -- how many articles are in this cluster  
  summary     TEXT,  
  first_seen  TIMESTAMPTZ DEFAULT NOW(),  
  last_seen   TIMESTAMPTZ DEFAULT NOW()  
);
```

alert_logs — External SIEM Webhook Ingestion

```
CREATE TABLE alert_logs (  
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  alert_source VARCHAR(100) NOT NULL,  -- Wazuh, Suricata, Snort, etc.  
  alert_type   VARCHAR(100) NOT NULL,  
  severity     VARCHAR(20) NOT NULL,  
  payload      JSONB NOT NULL,         -- raw alert payload  
  broadcast_status JSONB,              -- delivery receipts  
  created_at   TIMESTAMPTZ DEFAULT NOW()  
);
```

Performance Indexes

Index Name	Table	Column	Purpose
idx_threats_severity	threats	severity	Fast severity filter queries

Index Name	Table	Column	Purpose
idx_threats_published	threats	published_at DESC	Chronological timeline queries
idx_threats_category	threats	category	Category-based filtering
idx_threats_title_trgm	threats	title GIN trigram	Full-text keyword search
idx_mitre_tactic	threat_mitre_mapping	tactic_id	MITRE tactic filtering
idx_mitre_technique	threat_mitre_mapping	technique_id	MITRE technique filtering
idx_iocs_value	iocs	value	IOC search and deduplication

Row-Level Security (RLS)

All tables have PostgreSQL RLS enabled. Public read access is granted (needed for the frontend). Backend full read/write access is granted for the server connection pool. Alert logs are write-restricted to authenticated backend only.

Analytics Views

View	Query	Used By
v_severity_stats	COUNT threats grouped by severity	Dashboard severity pie chart
v_category_stats	COUNT threats grouped by category	Dashboard category breakdown
v_recent_threats	threats WHERE published_at >= NOW() - 7 days	Dashboard live stream + telemetry

SECTION 08

Ingestion Pipeline

The ingestion pipeline is an **automated scheduled process** that continuously collects fresh threat intelligence from all approved RSS/Atom feeds.

- **Frequency:** Every 30 minutes (configurable)
- **Trigger:** setInterval on server startup; or external cron job
- **Concurrency:** Per-host locking prevents duplicate parallel fetches

Deduplication (Two Levels)

1. **Database level:** `source_url` UNIQUE constraint — duplicate URLs fail silently on INSERT
2. **Cache level:** In-memory Set of known URLs prevents re-processing within same run

Run Telemetry

```
server/services/collectionRunService.js
```

Each collection run records: `runId`, `startedAt`, `completedAt`, `sourcesAttempted`, `sourcesSucceeded`, `articlesCollected`, `errors[]`. All returned by `GET /api/health`.

Admin Scripts

Script	Purpose
audit_existing_feeds.mjs	Checks all registered feeds and reports health status
build_canonical_registry.mjs	Builds the authoritative 1,000-source registry from candidates
verify_new_feeds.mjs	Tests new candidate feeds before adding to registry
populate_measured_health.mjs	Pre-fills health metrics for all sources
generate_source_permissions.mjs	Generates legal permission records for approved sources

SECTION 09

Source Registry & Legal Permissions

NO ENTRY maintains a curated registry of up to **1,000 approved** cybersecurity intelligence feeds. Each source goes through editorial and legal review before data from it is displayed.

Source Lifecycle

State	Description
Candidate	URL submitted; awaiting editorial review
Review	Feed verified, content quality assessed, legal permissions checked
Approved	Live — data collected every 30 minutes
Quarantined	Suspended due to quality issues or legal concern; under review
Rejected / Retired	Permanently removed from registry

Permission Record Structure

```
{
  "sourceId": "krebs-on-security",
  "permissionStatus": "permitted_intended_use",
  "permissionBasis": "RSS feed published with attribution requirement",
  "licenseType": "CC BY 4.0",
  "attributionRequired": true,
  "restrictions": {
    "maxSummaryLength": 500,           // max characters stored from RSS content
    "allowFullContentScrape": false,   // only use RSS snippet – no full scrape
    "allowAiEnrichment": true,         // AI may summarize and cluster this content
    "allowCommercialDisplay": false,   // research/personal use only
    "allowDownstreamRedistribution": false
  }
}
```

Source Categories

Category	Examples	Approx. Count
Government & CERT	CISA, NCSC, CERT-EU, US-CERT	~80
Vendor Security Labs	Microsoft MSRC, Mandiant, CrowdStrike, Sophos	~150
Threat Intelligence	Recorded Future, Malwarebytes, SANS ISC	~200
Security News	Krebs on Security, The Hacker News, BleepingComputer	~250
Academic & Research	IEEE, arXiv security, university CERTs	~100
CVE / Vuln Trackers	NVD, MITRE CVE, ExploitDB	~80
Community Blogs	Red Canary, PortSwigger, Schneier on Security	~140

SECTION 10

Deployment & Configuration

vercel.json

```
{
  "framework": "vite",
  "buildCommand": "cd soc-platform-ui-main && npm install && npm run build",
  "outputDirectory": "soc-platform-ui-main/dist",
  "rewrites": [
    { "source": "/api/(.*)", "destination": "/api" },
    { "source": "/((?!api|assets|soc-intel-.*|.*\\.\\.\\.).*)", "destination": "/index.html" }
  ]
}
```

Environment Variables

Variable	Required	Purpose
DATABASE_URL	Required	PostgreSQL connection string (Supabase or Neon)
VIRUSTOTAL_API_KEY	Optional	Enables IP/hash enrichment. Without it, enrichment returns graceful errors.
ABUSEIPDB_API_KEY	Optional	Enables IP abuse score lookup via AbuseIPDB
PORT	Dev Only	Local server port (default: 3000)
AI_SERVICE_URL	Optional	URL of Python AI microservice (default: http://localhost:8000)

⚠️ SERVERLESS LIMITATION

Vercel functions have a 10-60s execution limit. Long feed collection tasks are split into batches. For production-grade persistent collection, host the Node.js backend on Render or Railway (not Vercel).

SECTION 11

Complete REST API Reference

Node.js Backend

GET	/api/news	Latest aggregated threat articles. Params: limit, offset, severity, category, source, search.
GET	/api/explore	Date-range archive search. Params: preset, dateFrom, dateTo, tz, q, source, severity, tactic, technique, mappedOnly, page, pageSize, sort.
GET	/api/explore/meta	Filter options: all available sources, categories, MITRE tactics, severities, sort options.
GET	/api/mitre/news	Intelligence articles grouped by MITRE ATT&CK tactics and techniques. Params: tactic, technique, severity, search.
GET	/api/mitre/heatmap	All 14 tactics with technique hit counts for the frequency heatmap visualization.
GET	/api/sources	All registered sources with health metrics, permission status, and approval state.
GET	/api/enrich/ip/:ip	VirusTotal + AbuseIPDB enrichment for an IP address. Returns maliciousness score, country, ISP, and SIEM queries.
GET	/api/enrich/cve/:cveId	

FIRST EPSS probability score and CISA KEV exploitation status for a CVE identifier.

GET /api/enrich/queries/:ioc

Generates Splunk SPL, Microsoft Sentinel KQL, and Sigma YAML for any IOC.

GET /api/rules/sigma

Curated Sigma YAML detection rules with metadata. Params: search, level.

GET /api/rules/yara

Curated YARA malware signature rules. Params: search.

GET /api/ai/brief

Executive threat briefing (proxied to Python AI microservice). Returns headline, key threats, CVEs, actions, summary.

GET /api/ai/clusters

Clustered and deduplicated incidents with deduplication rate percentage.

GET /api/dashboard/snapshot

Telemetry: total articles, severity counts, top sources, isDemoEnabled flag.

GET /api/health

System health, uptime, version, last collection run telemetry.

POST /api/v1/alerts

External SIEM alert ingestion (requires Authorization header). Body: {alert_source, alert_type, severity, payload}.

Python AI Service APIs

Method	Endpoint	Request Body	Returns
GET	/health	—	{status: "healthy", uptime_check: true}
POST	/api/v1/briefing	{articles:[], focus_topic, max_bullet_points}	BriefingResponse with headline, threats, actions, summary
POST	/api/v1/cluster	{articles:[], similarity_threshold}	ClusterResponse with deduplication rate %
POST	/api/v1/remediate	{threat_type, severity, description, iocs}	4-phase incident response playbook
POST	/api/v1/sigma	{title, indicator, indicator_type, severity}	Sigma YAML + Splunk/KQL/Elastic queries

SECTION 12

Security & Legal

Control	Implementation	Protects Against
SSRF Protection	Block private IP ranges (RFC 1918) before fetch	Server-Side Request Forgery attacks
SSL Verification	Enforce TLS cert validation on all outbound fetches	Man-in-the-middle on feed collection
Content Size Limits	32MB max body; 500-char snippet cap for display	Memory exhaustion / DoS via large feeds
Row Level Security	PostgreSQL policies restrict writes to backend only	Unauthorized DB writes from stolen tokens
CORS Headers	Configured in vercel.json for all /api/* routes	Cross-origin request forgery
No Secrets in Code	All API keys via environment variables only	Credential exposure in version control
Alert Auth	SIEM webhook endpoint requires Authorization header	Unauthorized alert injection

 RSS LEGAL BASIS

NO ENTRY aggregates publicly available RSS feeds designed for syndication. The platform stores only snippets (max 500 chars), always links back to the original source, attributes the original author, and respects terms of service per source.

 LICENSE

The NO ENTRY platform source code is released under the **MIT License** — free to use, modify, and distribute, including commercially, with copyright notice included.

SECTION 13

Local Development Guide & File Structure

Prerequisites

Node.js v18+, Python 3.9+ (optional), Git, PostgreSQL database (Supabase free tier works well).

Setup Steps

```
# 1. Clone and enter project
git clone https://github.com/ceylonroameryt-bit/SOC-AI.git && cd SOC-AI

# 2. Install frontend + backend dependencies
cd soc-platform-ui-main && npm install

# 3. Environment variables (.env in project root)
PORT=3000
DATABASE_URL=postgresql://postgres:[pwd]@db.supabase.co:5432/postgres
VIRUSTOTAL_API_KEY=your_key_here
ABUSEIPDB_API_KEY=your_key_here

# 4. Initialize database schema
psql $DATABASE_URL < schema.sql

# Terminal 1 - Node.js Backend (port 3000)
npm run server

# Terminal 2 - React Frontend (port 5174)
npm run dev

# Terminal 3 - Python AI Service (port 8000, optional)
pip install -r requirements.txt && python app.py
```

Project File Structure

```
SOC-AI/
├─ api/index.js      ← Vercel serverless bridge
├─ app.py            ← Python FastAPI AI microservice
├─ schema.sql        ← PostgreSQL database schema
├─ vercel.json       ← Vercel deployment configuration
```

```

├─ requirements.txt      ← Python dependencies
├─ scripts/              ← Admin & pipeline management scripts
│   ├── audit_existing_feeds.mjs
│   ├── build_canonical_registry.mjs
│   ├── verify_new_feeds.mjs
│   ├── generate_source_permissions.mjs
│   └─ generate_all_reports.py
├─ soc-platform-ui-main/
│   ├── src/
│   │   ├── App.tsx      ← Root app with all routing
│   │   ├── config/api.ts ← API_BASE URL configuration
│   │   ├── context/
│   │   │   └─ AccessibilityContext.tsx ← Screen reader announcements
│   │   ├── pages/       ← 15 page components
│   │   │   ├── Dashboard.tsx      (274 lines)
│   │   │   ├── Explore.tsx        (1,085 lines – most complex)
│   │   │   ├── Sources.tsx        (964 lines)
│   │   │   ├── Enrichment.tsx     (477 lines)
│   │   │   ├── MitreHeatmap.tsx   (312 lines)
│   │   │   ├── MitreNews.tsx      (large)
│   │   │   ├── AIBrief.tsx        (311 lines)
│   │   │   ├── RuleLibrary.tsx    (335 lines)
│   │   │   └─ ... (7 more pages)
│   │   └─ components/
│   │       ├── dashboard/         ← Dashboard widgets
│   │       │   ├── TelemetryCards.tsx
│   │       │   ├── NewsFeed.tsx
│   │       │   ├── SeverityChart.tsx
│   │       │   └─ MitreMiniMatrix.tsx
│   │       └─ AiExecutiveWidget.tsx
│   │   └─ layout/                 ← Navigation & app shell
│   └─ server/
│       ├── server.js              ← Express server entry point
│       ├── routes/
│       │   ├── news.js
│       │   ├── explore.js         ← Timezone-aware archive search
│       │   ├── mitre.js
│       │   ├── sources.js
│       │   └─ enrich.js
│       └─ services/
│           ├── collectorEngine.js ← RSS feed ingestion with host locking
│           ├── severityEngine.js  ← Multi-factor keyword scoring
│           ├── mitreService.js    ← ATT&CK technique mapping
│           └─ collectionRunService.js ← Run telemetry

```

SECTION 14

Glossary

Term	Full Name	Definition
APT	Advanced Persistent Threat	A sophisticated, long-term cyberattack, often state-sponsored
ATT&CK	Adversarial Tactics, Techniques & Common Knowledge	MITRE's framework documenting adversary behavior patterns
CERT	Computer Emergency Response Team	An organization handling cybersecurity incidents for a region or sector
CISA	Cybersecurity and Infrastructure Security Agency	U.S. government agency responsible for national cybersecurity
Cosine Similarity	—	Measure of similarity between two text vectors (0 = different, 1 = identical)
CVE	Common Vulnerabilities and Exposures	Industry-standard ID for public security vulnerabilities (e.g. CVE-2024-12345)
EPSS	Exploit Prediction Scoring System	Daily probability score predicting if a CVE will be exploited in the wild
HMR	Hot Module Replacement	Vite feature that applies code changes instantly without full page refresh
IOC	Indicator of Compromise	Observable evidence of a cyberattack: IP addresses, file hashes, domains, CVEs
KEV	Known Exploited Vulnerabilities	CISA's catalog of CVEs actively exploited by real threat actors
KQL	Kusto Query Language	Microsoft's query language for Azure Sentinel SIEM and Log Analytics
MITRE	MITRE Corporation	Non-profit that maintains the ATT&CK framework and CVE list
RLS	Row Level Security	PostgreSQL feature restricting which rows each database role can access

Term	Full Name	Definition
RSS	Really Simple Syndication	XML format for publishing regularly updated content (news feeds)
SIEM	Security Information and Event Management	Platform aggregating and analyzing security logs (Splunk, Sentinel, Elastic)
Sigma	—	Platform-agnostic YAML detection rule format convertible to any SIEM query language
SOC	Security Operations Center	Team of analysts monitoring, detecting, and responding to cybersecurity incidents
SPA	Single Page Application	Web app that loads once and updates content dynamically without full page reloads
SPL	Search Processing Language	Splunk's query language for searching and analyzing log data
SSRF	Server-Side Request Forgery	Attack tricking a server into making requests to internal or unauthorized systems
Tactic	—	MITRE ATT&CK: high-level attacker goal (e.g. Initial Access, Persistence, Exfiltration)
Technique	—	MITRE ATT&CK: specific method to achieve a tactic (e.g. T1486 — Data Encrypted for Impact)
TF-IDF	Term Frequency-Inverse Document Frequency	Statistical weight measuring word importance in a document vs. a collection
TTP	Tactics, Techniques, and Procedures	The behavior patterns of a threat actor, described using ATT&CK framework
YARA	Yet Another Ridiculous Acronym	Tool for creating malware detection rules based on binary file patterns
Zero-Day	—	Vulnerability unknown to the vendor with no patch — exploitable from day zero

**NO ENTRY Platform**

Built with ❤️ for the Cybersecurity and Blue Team Community

soc-ai-six.vercel.app · github.com/ceylonroameryt-bit/SOC-AI · MIT License